

• Easy • #35 • Search Insert Position

▲ Problem Statement → Given a sorted array & a target value, return its index if found. Otherwise, return the index where it should be inserted.

▲ Main Solution: Use "Binary Search". Since the array is sorted, repeatedly check the middle element & search only the required half.

▲ Brute Force: Loop through the array & find the first position where the target is less than or equal to the current element.

▲ Algorithm: 1. Set "left = 0" & "right = nums.size() - 1"

2. While "left <= right": - Find the middle index.

• If "nums[mid] == target", return "mid".

• If "nums[mid] < target", search the right half.

• Otherwise, search the left half.

3. If the target is not found, "left" will be correct insertion position.

4. Return "left".

▲ Important Points: - The array is sorted.

• Binary Search gives " $O(\log n)$ " time complexity.

• If the target is not found, "left" becomes the correct insertion ~~index~~ index.

• Use "left + (right - left) / 2" to safely calculate "mid".

M	T	W	T	F	S	S
Page No.:					YOUVA	
Date:						

 Complexity → Brute Force → $O(n)$ & $O(1)$
 Binary Search → $O(\log n)$ & $O(1)$